

Scientific Computation

Prof. Matthew Giamou
Nov. 27, 2025

Lecture 31 - Solving ODEs: Stiffness and Runge-Kutta

Lecture Outline

- Stiffness
- Runge-Kutta methods

Numerical “Stiffness”

Sometimes we have systems of ODEs that contain functions that act on wildly different timescales. This poses the following problem:

→ The faster changing components require a small time step h for explicit method to be stable

→ Implicit methods are slower in general

→ The slower components take a long time to settle, so a longer time horizon is needed ($\Rightarrow$ more steps)

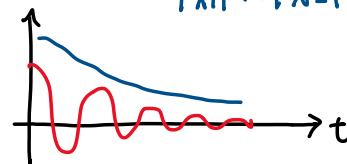
3

The General Linear Case

Let's look at the case of a $n \times n$ system of linear ODEs:

$$y' = Ay,$$

where $A \in \mathbb{R}^{n \times n}$ is diagonalizable.



$\Rightarrow A = P\Lambda P^{-1}$, where Λ diag [eigenvalues] of A

$\Rightarrow y' = Ay = (P\Lambda P^{-1})y \Rightarrow P^{-1}y' = \Lambda(P^{-1}y)$

Let $z \triangleq P^{-1}y \Rightarrow z' = \Lambda z$. Because Λ is diag. we have n uncoupled ODEs

$$z_i' = \lambda_i z_i \quad (z(t) = e^{\lambda_i t} z_i(0)) \quad \text{stiffness ratio} \triangleq \frac{\max_i |\lambda_i|}{\min_i |\lambda_i|} \quad \left. \vphantom{\frac{\max_i |\lambda_i|}{\min_i |\lambda_i|}} \right\} \text{essentially } \text{Cond}_2(A)$$

4

Implicit Methods Preferred

Recalling our earlier example for $y' = \lambda y$, we showed that stability of the explicit forward Euler method required

$$h \leq \frac{2}{|\lambda|}.$$

Contrastingly, implicit methods are inherently stable for any h

But takes longer time.

5

What About Nonlinearity?

So far our discussion of stability and stiffness has been restricted to linear problems, but in general we want to solve nonlinear ODEs.

We can analyze by linearizing the non linear system

Then we can study the eigs/stiffness of this local linear approx.

Lecture Outline

- Stiffness
- Runge-Kutta methods

Runge-Kutta Methods

These time-discretizing numerical methods for solving ODEs that we have been studying are all called **Runge-Kutta** methods and were developed around the turn of the 20th century (~1900).

→ Forward and backward Euler are RK methods (1st. order)

(... and with explicit Euler are 1st methods (1st order))

→ more RK methods, both implicit and multistage explicit methods.

8

Implicit Trapezoidal Rule

Recall the implicit numerical method we obtained by using the **trapezoidal rule** to approximately solve $y' = f(t, y(t))$:

$$y_{i+1} = y_i + \frac{h}{2}(f(t_i, y_i) + f(t_{i+1}, y_{i+1})).$$

This came from

$$y(t_{i+1}) = y(t_i) + \int_{t_i}^{t_{i+1}} f(t, y(t)) dt$$

9

Error of the Implicit Trapezoidal Method

To analyze the error of this method, we need to write out two Taylor series expansions, one about t_i , and the other about t_{i+1} :

Recall the local truncation error,

$$d_i \triangleq \frac{y(t_{i+1}) - y(t_i)}{h} - \frac{1}{2} [f(t_i, y(t_i)) - f(t_{i+1}, y(t_{i+1}))]$$

$$y(t_{i+1}) \stackrel{①}{=} y(t_i) + h y'(t_i) + \frac{h^2}{2} y''(t_i) + \frac{h^3}{6} y'''(\eta_i), \quad \eta_i \in [t_i, t_{i+1}]$$

$$\begin{aligned} y(t_i) &\stackrel{②}{=} y(t_{i+1}) + (t_i - t_{i+1}) y'(t_{i+1}) + \frac{h^2}{2} y''(t_{i+1}) + \frac{1}{6} (t_i - t_{i+1})^3 y'''(\xi_i) \\ &= y(t_{i+1}) - h y'(t_{i+1}) + \frac{h^2}{2} y''(t_{i+1}) + \frac{1}{6} h^3 y'''(\xi_i) \end{aligned}$$

10

Error of the Implicit Trapezoidal Method Cont.

Subtracting the expressions for $y(t_{i+1})$ and $y(t_i)$ gives:

$$\begin{aligned} [y(t_{i+1}) - y(t_i)] &= [y(t_i) - y(t_{i+1})] + h [y'(t_i) + y'(t_{i+1})] + \frac{h^2}{2} [y''(t_i) - y''(t_{i+1})] + \\ &\quad \frac{h^3}{6} [y'''(\eta_i) + y'''(\xi_i)] \end{aligned}$$

$$\begin{aligned} \Rightarrow y(t_{i+1}) - y(t_i) &= \frac{h}{2} (y'(t_i) + y'(t_{i+1})) + \frac{h^2}{4} [y''(t_i) - y''(t_{i+1})] \\ &\quad + \frac{h^3}{12} [y'''(\eta_i) + y'''(\xi_i)] \end{aligned}$$

$$d_i = \frac{1}{2} [y'(t_i) + y'(t_{i+1}) - f(t_i, y(t_i)) - f(t_{i+1}, y(t_{i+1}))] + \frac{h}{4} [y''(t_i) - y''(t_{i+1})] + \frac{h^2}{12} [y'''(\eta_i) + y'''(\xi_i)]$$

11

Applying the MVT

We can transform the second-order terms as follows:

$$\forall \tau: \exists \tau_i \in [t_i, t_{i+1}] \text{ , s.t. } y'''(\tau_i) = \frac{y''(t_i) - y''(t_{i+1}))}{h}$$

$$\Rightarrow y''(t_{i+1}) - y''(t_i) = h \cdot y'''(\tau_i)$$

$$\begin{aligned} \Rightarrow d_i &= \frac{h}{4} (-h) y'''(\tau_i) + \frac{h^2}{12} [y'''(\eta_i) + y'''(\xi_i)] \\ &= h^2 \left[\frac{y'''(\eta_i) + y'''(\xi_i)}{2} - \frac{1}{4} y'''(\tau_i) \right], \end{aligned}$$

which is $O(h^2)$ and therefore 2nd order.

12

Another Approach: Quadrature Error

Alternatively, we can recall the error of the trapezoidal rule for quadrature:

$$I_f - I_{\text{trap}} = -\frac{f''(\eta)}{12}(b-a)^3, \quad \eta \in (a, b).$$

$$f = y'(t) = d_i = \frac{-y'''(\eta) h^2}{12}, \text{ which is also } O(h^2)$$

13

An Explicit Method from the Trapezoidal Approach

We know that implicit methods are stable, but sometimes we need the speed that an explicit method brings. Let's return to our implicit trapezoidal update rule and take a different approach:

$$y_{i+1} = y_i + \frac{h}{2}(f(t_i, y_i) + f(t_{i+1}, y_{i+1})). \quad \textcircled{1}$$

Stage 1: Approximate $y(t_{i+1})$ with forward Euler:

$$y(t_{i+1}) \approx y \triangleq y_i + h \cdot f(t_i, y_i)$$

Stage 2: Use y to make $\textcircled{1}$ explicit:

$$y_{i+1} = y_i + \frac{h}{2}[f(t_i, y_i) + f(t_{i+1}, y)]$$

This is a 2 stage explicit RK method (RK2)

The local truncation error is $O(h^2)$

14

Explicit 2-Stage RK Performance

Example 1: Let's try this explicit method on $y' = \lambda y$ as usual.

$$\text{Stage 1: (F.E.) } y = y_i + h\lambda y_i = (1 + h\lambda)y_i$$

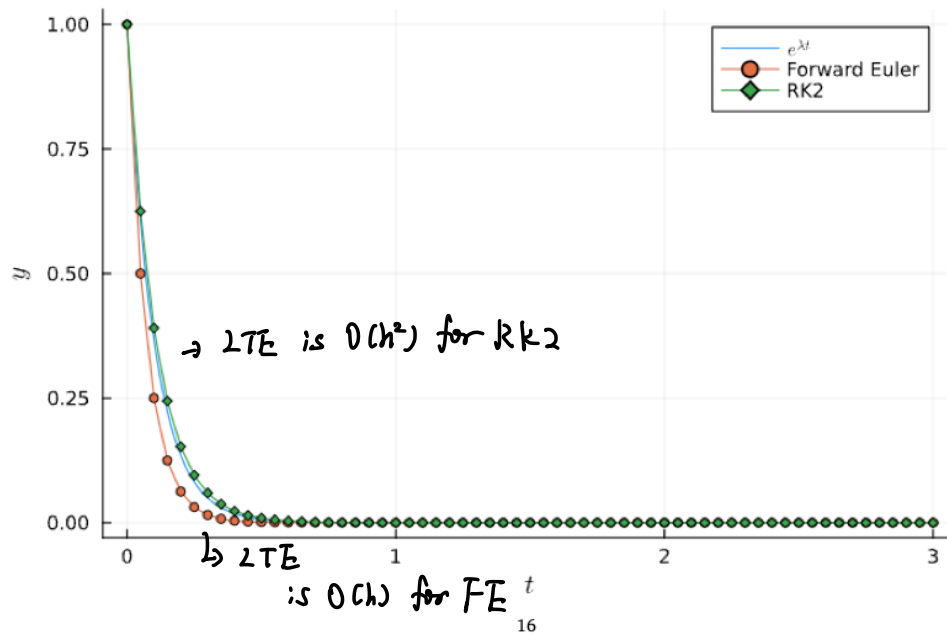
$$\begin{aligned} \text{Stage 2: } y_{i+1} &= y_i + \frac{h}{2}[\lambda y_i + \lambda y] = y_i + \frac{h}{2}[\lambda y_i + \lambda(1 + h\lambda)y_i] \\ &= (1 + h\lambda + \frac{h^2\lambda^2}{2})y_i \end{aligned}$$

Analyze this for stability $\frac{h^2\lambda^2}{2} > 0$

RK2 vs. Forward Euler (Accuracy)

Example 1 cont.: For $\lambda = -10$, $y(0) = 1$ and $h = 0.05 < 0.2$ we get:

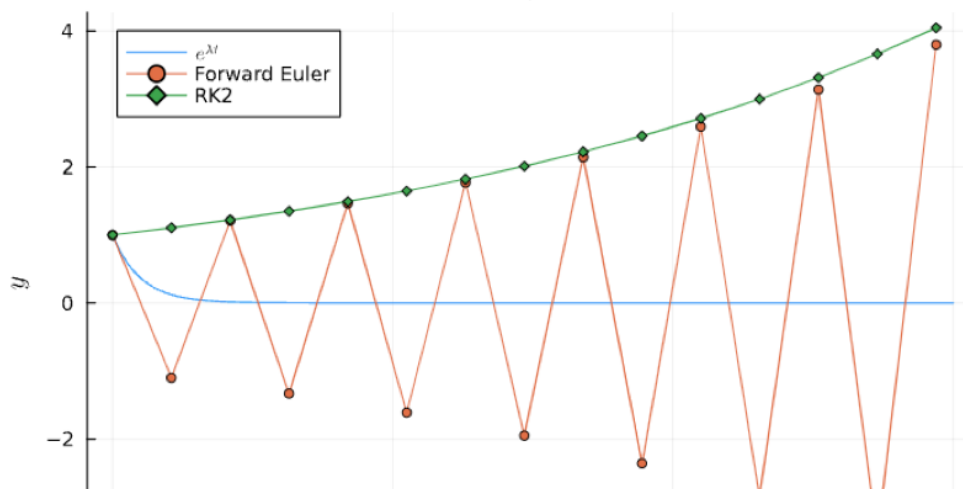
$$\lambda = -10, h = 0.05$$

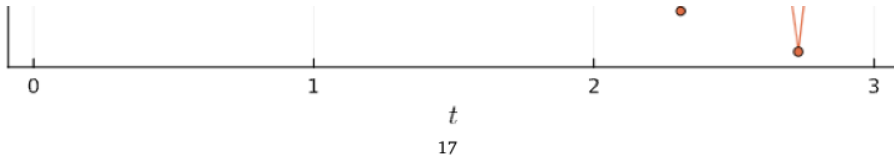


RK2 vs. Forward Euler (Stability)

Example 1 cont.: For $\lambda = -10$, $y(0) = 1$ and $h = 0.21 > 0.2$ we get:

$$\lambda = -10, h = 0.21$$





The Midpoint Rule

What if we applied the midpoint rule instead of the trapezoidal rule? The implicit update rule would become

$$y_{i+1} = y_i + h \cdot f(t_i + h/2, (y_i + y_{i+1})/2).$$

Can also get a 2-stage explicit rule:

$$\text{Stage 1: } y \triangleq y_i + \frac{h}{2} \cdot f(t_i, y_i) \approx \frac{y_i + y_{i+1}}{2}$$

$$\text{Stage 2: } y_{i+1} = y_i + h \cdot f(t_i + \frac{h}{2}, y)$$

Also a 2nd order method (LTE is $O(h^2)$)

18

Fourth-Order Runge-Kutta (RK4)

We will not derive it in this course, but one very popular RK method is the following fourth-order (i.e., $O(h^4)$ truncation error) rule based on Simpson's quadrature rule:

$$\text{Let } y_1 = y_i$$

$$\text{stage 1: } y_2 = y_1 + \frac{h}{2} \cdot f(t_i, y_1)$$

$$\text{stage 2: } y_3 = y_1 + \frac{h}{2} \cdot f(t_i + \frac{h}{2}, y_2)$$

$$\text{stage 3: } y_4 = y_1 + h \cdot f(t_i + \frac{h}{2}, y_3)$$

$$\text{stage 4: } y_{i+1} = y_1 + \frac{h}{6} [f(t_i, y_1) + 2 \cdot f(t_i + \frac{h}{2}, y_2) + 2 \cdot f(t_i + \frac{h}{2}, y_3) + f(t_{i+1}, y_4)]$$

Stiffness 就是指一个DE包含发生的极快和极慢的过程，因为用forward method的时候，h被限制的很小，但是总距离T很长，所以用forward method在这里不如backward。

现在我们推广标量微分方程到线性方程组，即矩阵形式。

注意，这里O(h)是指error减小的速率，和步长h正相关。

我们现在总结一下数值解ODE方法：

一、显式方法 (Explicit Methods)

特点： 下一步的状态 y_{n+1} 仅仅依赖于当前已知的状态 y_n 。计算简单，速度快，但如果步长 h 太大容易不稳定（爆炸）。

1. 前向欧拉法 (Forward Euler)

这是最简单的方法，本质上是“左矩形积分”的应用。

- **计算公式：**

$$y_{n+1} = y_n + h \cdot f(t_n, y_n)$$

- **如何算：**

1. 算出当前点的斜率 $k = f(t_n, y_n)$ 。
2. 直接用这个斜率往前走一步长 h 。

2. 改进欧拉法 / 赫恩法 (Heun's Method / RK2)

这是一个“预测-校正”方法，属于二阶龙格-库塔法 (RK2)。

- **计算公式：**

$$y_{n+1} = y_n + \frac{h}{2} [f(t_n, y_n) + f(t_{n+1}, \tilde{y}_{n+1})]$$

- **如何算：**

1. **预测 (Predict)：** 先用前向欧拉法猜一个终点值 $\tilde{y}$ 。

$$\tilde{y} = y_n + h \cdot f(t_n, y_n)$$

2. **校正 (Correct)：** 计算起点斜率 k_1 和预测终点的斜率 $k_2 = f(t_{n+1}, \tilde{y})$ ，然后取平均值来

史和 y 。

3. 四阶龙格-库塔法 (RK4)

这是工业界的标准方法，精度高（四阶精度），稳定性比欧拉法好。

- **如何算（加权平均）：** 每一步需要计算 4 个斜率：

1. $k_1 = f(t_n, y_n)$ (起点斜率)
2. $k_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_1)$ (中点预测斜率1)
3. $k_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}k_2)$ (中点预测斜率2)
4. $k_4 = f(t_n + h, y_n + hk_3)$ (终点预测斜率)

最终更新：

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

二、隐式方法 (Implicit Methods)

特点： 下一步的状态 y_{n+1} 出现在方程的等号两边。你必须**解方程**才能算出 y_{n+1} 。通常用于解决**刚性 (Stiff)** 问题。

1. 后向欧拉法 (Backward Euler)

虽然也是一阶精度，但它是无条件稳定的（对于线性衰减系统）。

- **计算公式：**

$$y_{n+1} = y_n + h \cdot f(t_{n+1}, y_{n+1})$$

- **如何算：**

- 如果是线性方程 $y' = Ay$ ，公式变为 $y_{n+1} = y_n + hAy_{n+1}$ ，可以直接解出：

$$y_{n+1} = (I - hA)^{-1}y_n$$

- 如果是非线性方程，需要使用牛顿迭代法求根。

2. 梯形公式 (Trapezoidal Rule)

精度比后向欧拉高（二阶），也是隐式的。

- **计算公式：**

$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$

- **如何算：** 同样需要移项求解或者迭代求解。